

Network Traffic Anomaly Detector

Personal Research Project -Research Report

Real-time machine-learning intrusion detection with a live attack-and-defend lab



Author: Angel Rusev

Author	Angel Rusev
Programme	BSc ICT & Software Engineering
Minor	Cybersecurity -Attack & Defend
Institution	Fontys University of Applied Sciences
Semester	Spring 2026
Document	Research Report v2.0 -supersedes PRP v1.0 (March 2026)
Repository	https://github.com/Guts1313/AnomalyDetector

Table of contents

Table of contents	2
Abstract	4
Foreword	4
Part I -Framing and planning	5
1. The problem and the question	5
2. Project planning and Scrum delivery.....	6
3. A methodological compass: the DOT framework.....	8
4. System architecture.....	9
5. Data and feature engineering (SRQ1)	12
6. Algorithm selection (SRQ2)	14
7. Real-time performance (SRQ3)	15
8. False-positive control (SRQ4)	16
9. Per-category performance (SRQ6)	16
Part III -Attack and defend	18
10. The analyst interface (SRQ5)	18
11. The attack/defend laboratory	26

12. Offensive security: threat model and risk analysis.....	31
13. Defensive security: requirements, monitoring and response	33
Part IV -Synthesis.....	35
14. Triangulation: do the perspectives agree?	35
15. Limitations and honest caveats.....	35
16. Future work	36
17. Professional practice, ethics and reflection	36
18. Conclusion	38
Appendix E — The dark theme.....	39
References	43

Abstract

This report documents the design, construction and evaluation of a machine-learning network-traffic anomaly detector, built as the Personal Research Project (PRP) for the Cybersecurity Attack & Defend minor. The system learns the statistical shape of benign network flows and flags deviations as one of seven attack categories -denial-of-service, distributed denial-of-service, port scanning, brute force, web attack, botnet and infiltration -in real time and at a controlled false-positive rate. The research follows the Fontys Development-Oriented Triangulation (DOT) framework, triangulating a literature study, a reproducible laboratory benchmark, a field-informed analyst interface and a live demonstration. A Gradient Boosting model selected from a four-algorithm benchmark reaches 0.99 macro-F1 on six of eight classes, holds the operator-visible false-positive rate below one per cent, and predicts in under seven microseconds per flow. Beyond the offline evaluation, the artefact is exercised inside a containerised attack-and-defend laboratory: an attacker container drives real offensive tooling (nmap, hping3, hydra, sqlmap) against a victim, a defender container captures the traffic, extracts flow features, classifies them with the trained model, and installs an automatic firewall block -closing the loop from real packets to machine verdict to defensive response. The report substantiates each of the six sub-research questions, presents a STRIDE threat model and an Impact $\times$ Likelihood risk register for the detector itself, and reports honestly on the one category -infiltration -where flow-level statistics are intrinsically blind.

Keywords: anomaly detection, intrusion detection, CICIDS2017, Gradient Boosting, Isolation Forest, false-positive control, FastAPI, attack/defend lab, STRIDE, DOT framework.

Foreword

This report is the research deliverable of the Personal Research Project (PRP) for the Cybersecurity Attack & Defend minor. Where the PRP itself was a proposal, this document tells the story of the project that was actually built, evaluated, and reasoned about. Every claim it makes is anchored either in a piece of literature or in an experiment that can be reproduced in less than two minutes from the accompanying repository. The objective is not to prove that anomaly detection works -that has been settled for decades -but to design a defensible answer to the question of how to do it well, in real time, without drowning a security analyst in false positives, and to demonstrate that the resulting model is not merely a spreadsheet artefact but a component that can sit in a real detect-and-respond pipeline.

The document is organised in four parts. Part I frames the problem and lays out how the work was planned and delivered across four Scrum sprints. Part II is the engineered artefact: its architecture, its data and features, its model, and its performance. Part III is the heart of the

Attack & Defend minor -the live laboratory, the offensive threat model and the defensive controls. Part IV synthesises the evidence, names the limitations, and concludes.

Part I -Framing and planning

1. The problem and the question

Modern organisations process more network traffic in an hour than a SOC analyst can manually scrutinise in a year. Signature-based intrusion-detection systems such as Snort and Suricata help, but they are by construction unable to recognise an attack whose signature has never been seen before. Anomaly-based detection offers a complementary perspective: rather than asking what malicious traffic looks like, it learns what normal traffic looks like and flags any deviation. Its weakness is well documented -false-positive rates that turn an analyst's console into noise -but its potential to surface zero-day and novel behaviour is exactly what the signature approach cannot deliver.

The economics of that weakness are worth stating plainly. An analyst who must triage a queue can sustain only a finite alert rate before genuine incidents are lost in the noise; the published SOC literature repeatedly identifies alert fatigue, not detection capability, as the binding constraint. A detector that is ninety-nine per cent accurate but raises thousands of false alarms a day is, in operational terms, worse than a less sensitive detector that an analyst can actually keep up with. This is why the research question below treats the false-positive rate as a first-class design target rather than an afterthought, and why two of the six sub-questions are devoted to controlling it.

The PRP for this project framed that tension in a single main research question:

How can a machine-learning-based network-traffic anomaly detector be designed and implemented to accurately identify cyber attacks in real time while maintaining an acceptable false-positive rate?

Six sub-research questions decompose that ambition into pieces that can be investigated independently -features, algorithms, latency, tuning, presentation, and per-category performance. They are summarised in Table 1 and each is taken up in turn in Part II.

SRQ	Question (abridged)	DOT strategies
SRQ1	Which flow features are most indicative of anomalous behaviour and how should they be extracted?	Library + Lab
SRQ2	Which ML algorithms balance accuracy, speed and false-positive rate best?	Library + Lab + Showroom

SRQ3	How can the system run in real time without unacceptable latency?	Lab + Workshop
SRQ4	What threshold and tuning strategies minimise false positives?	Lab
SRQ5	How should anomalies be presented to analysts in an actionable way?	Library + Field + Showroom
SRQ6	How does the detector perform across the different attack categories?	Lab

Table 1 -Sub-research questions and the DOT strategies used to investigate them.

2. Project planning and Scrum delivery

The project was delivered in four two-to-four-week Scrum sprints between March and June 2026, each closing with a concrete hand-in tied to a phase of the Fontys applied-research lifecycle. Sprint 1 covered Define & Analyse, Sprint 2 Design & Implement, Sprint 3 Optimise & Validate, and Sprint 4 the research write-up, presentation and final delivery. Figure 1 places every work item on a timeline against the four milestone hand-in dates; the diamonds mark the sprint deadlines.



Figure 1 -Gantt chart of the PRP. Each bar is a work item, colour-coded by the sprint it belongs to; red diamonds mark the four deliverable hand-in dates (17 March, 26 March, 13 April, 11 May) and the 11 June final delivery.

Working in fixed-length sprints had two concrete benefits for a solo project. First, it forced a deliverable at the end of every sprint rather than a single big-bang submission, so the

architecture review in Sprint 2 could still change the model-selection rule before it was frozen in Sprint 3. Second, it kept the research and the engineering interleaved: the literature study that answered SRQ1 and SRQ2 ran in Sprint 1, but the lab benchmark that confirmed it ran in Sprint 2, so each leg of the triangulation had time to contradict the other before the report committed to a conclusion. Table 2 maps each sprint to the deliverables it produced and where they live in the repository.

Sprint / date	Lifecycle phase	Key deliverables produced	Where
Kick-off · 17 Mar	Idea & definition	Project idea, main + six sub-research questions, scope and out-of-scope list.	PRP v1.0
Sprint 1 · 26 Mar	Define & Analyse	Project plan, user stories, Scrum board, literature study / trend analysis, DOT method matrix.	GitHub Projects, -3–4
Sprint 2 · 13 Apr	Design & Implement	Feature schema, synthetic dataset generator, 4-algorithm benchmark, FastAPI proof-of-concept, network drawing, flowcharts, attack scenarios, technical design.	src/, scripts/, -5–7
Sprint 3 · 11 May	Optimise & Validate	Threshold/severity tuning, per-class evaluation, test suite, React + Streamlit dashboards, attack/defend lab, coach feedback round.	tests/, frontend/, lab/, -8–14
Sprint 4 · 11 Jun	Deliver	Research document, advisory report, presentation slides; intermediate research results presented to coach and group.	docs/Research_Report.docx, docs/*.pptx

Table 2 -Sprint deliverables mapped to the applied-research lifecycle and the repository.

2.1 Stakeholders and traceable goals

Even a solo project has stakeholders, and naming them keeps the work honest about who it is for. Four were identified and involved through the channels listed in Table 3: the Fontys supervisor, who reviews methodological rigour through the GitHub issues that track each phase; a hypothetical SOC analyst, whose needs were modelled as a persona and checked against published SIEM conventions; a peer student, whose role is realised as the workshop-style architecture review; and the eventual NOC operator, for whom the response runbooks are written in plain operational language. Each project goal is traceable from the main research goal down to a concrete engineering or operational target, so that no feature exists without a reason that points back to the research question.

Stakeholder	Interest	How involved
Supervisor (Fontys)	Methodological rigour, PRP fidelity	Reviews the DOT document; comments on the per-phase GitHub issues.
Hypothetical SOC analyst	Dashboard usability, severity model	Persona modelling for SRQ5; checked against SIEM UI conventions.
Peer student / reviewer	Code-review feedback	Workshop-style architecture-review checklist (-4.1).
NOC operator (end user)	Reliability, runbooks	Procedural runbooks written in operator language (-13).

Table 3 -Stakeholders and how each was involved.

3. A methodological compass: the DOT framework

The Development-Oriented Triangulation (DOT) framework prescribed by Fontys groups applied-research activities into five method strategies -Library, Lab, Field, Workshop and Showroom -and asks the researcher to triangulate conclusions across at least three of them. Triangulation is the antidote to the most common failure mode in applied research: drawing a strong conclusion from a single, possibly biased, source of evidence.

For this project Library represents the literature study and the survey of existing intrusion-detection tools. Lab is the bulk of the work -feature engineering, training, benchmarking, latency measurement -all performed against a reproducible synthetic CICIDS-compatible dataset, and later extended into the live containerised attack/defend laboratory of Part III. Field consists of the SOC-analyst persona modelling that shaped the dashboard, informed by publicly documented SIEM usability conventions. Workshop is the peer-style architecture review that scored every design decision against alternatives. Finally, Showroom is the live demonstration of the working system, which the screenshots throughout this report stand in for. Each sub-research question draws on at least two of these strategies, and the main research question is answered from three independent angles in Part IV. Table 4 is the concrete instantiation of the framework for this project.

Strategy	Concrete methods used	Applied to	Artefacts
Library	Literature study (Sharafaldin 2018; Liu 2008; Chandola 2009; Ring 2019); product analysis (Snort/Suricata/Zeek); framework docs (scikit-learn, FastAPI).	SRQ1, SRQ2, SRQ5	-4-5, References
Lab	Synthetic CICIDS generator; 4-algorithm benchmark; ROC + threshold sweep; latency micro-benchmark; per-class	SRQ1-SRQ6	models/, lab/, -5-

	evaluation; live attack/defend lab.		12
Field	SOC-analyst persona modelling; review of SIEM UI conventions (Splunk, Elastic, Wazuh).	SRQ5	frontend/, dashboard/
Workshop	Peer-style architecture review; code-review checklist on the API surface.	SRQ3	docs/architecture/
Showroom	Live demonstration of the working detector; peer review via GitHub issues.	All SRQs	Repo, screenshots

Table 4 -DOT method matrix: the concrete methods used per strategy.

Part II -The engineered artefact

4. System architecture

Before the research questions are answered one by one, it helps to see the shape of the thing they were answered against. The detector is a single-purpose internal analytic for a hypothetical Network/Security Operations Centre. Figure 2 is the C4 level-1 context diagram: offline PCAP or CSV captures are turned into flow features and scored; a security analyst consumes the resulting alerts; an ML engineer periodically retrains and reloads the model; and a severity feed can be forwarded to an upstream SIEM.

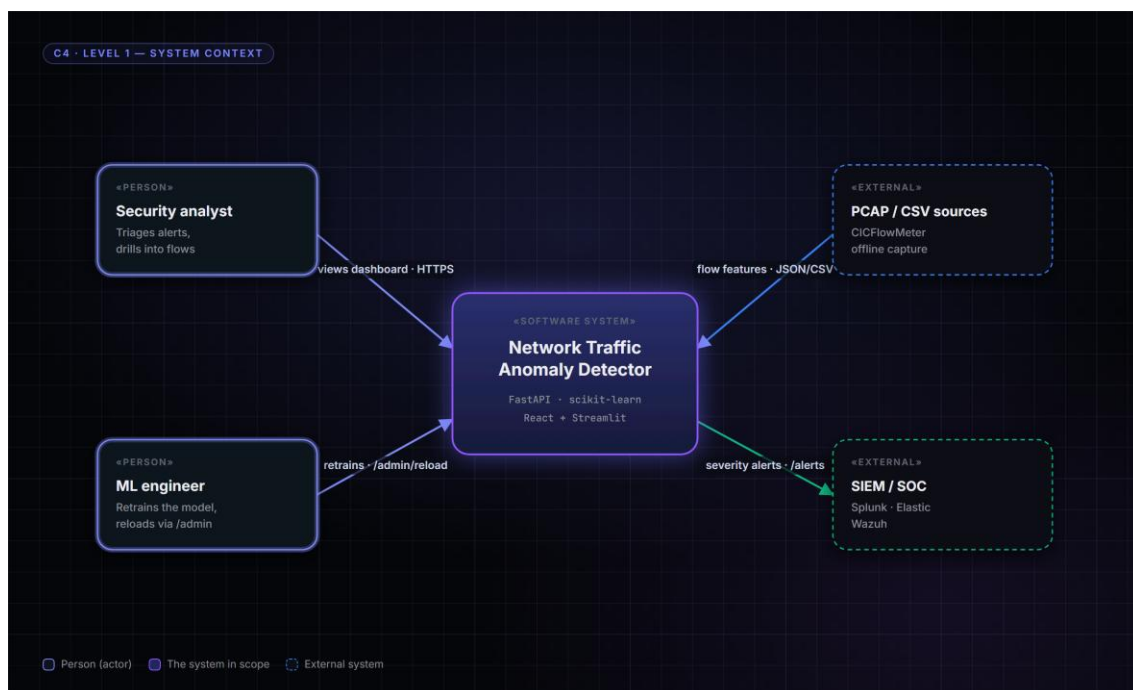


Figure 2 -C4 level-1 context. The detector sits between flow-feature sources and the two human roles it serves -the analyst who triages alerts and the ML engineer who maintains the model -and can forward severity alerts to an existing SIEM.

Figure 3 opens the box one level further. The inference core is a stateless FastAPI service that loads a single joblib bundle containing both the fitted preprocessing pipeline and the model, and writes every verdict to a SQLite audit store. Two presentation surfaces sit in front of it -a polished React single-page application and a lighter Streamlit dashboard -and, for the Attack & Defend minor, two laboratory containers sit behind it: an attacker that generates real offensive traffic and a defender that captures, classifies and blocks it.

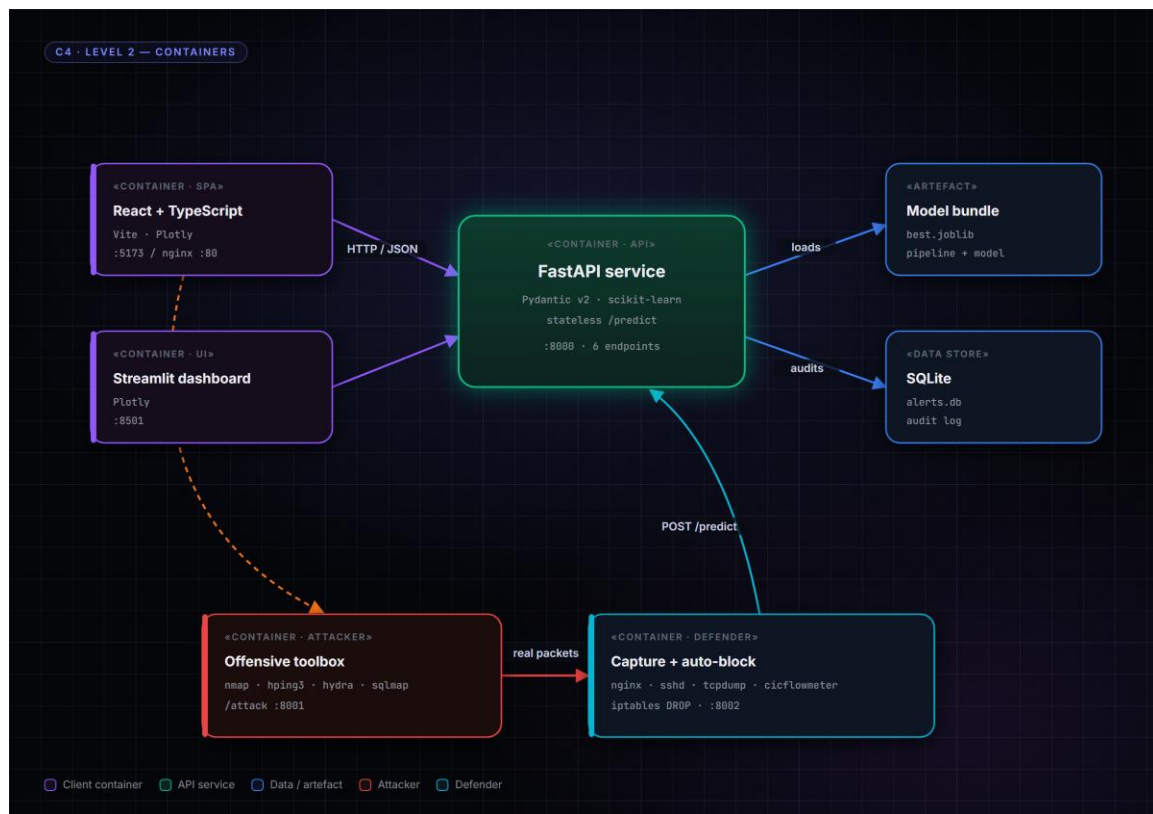


Figure 3 -C4 level-2 container view. The FastAPI service is the hub; the React SPA and Streamlit dashboard are clients; the model bundle and SQLite store are dependencies; and the attacker/defender containers form the live lab that drives traffic through the same /predict endpoint.

Three architectural rules were fixed early and held for the rest of the project, because each one removes a documented source of false positives or operational fragility. The /predict endpoint is stateless -no per-request database round-trip sits on the critical path; the audit write happens after the verdict is computed. The fitted preprocessor and the model travel together in one serialised bundle, which guarantees that the exact transformations applied at training time are

applied at inference time, eliminating train/serve skew. And the endpoint accepts a batch of flows by default, so the vectorised scikit-learn predict path does the work instead of a Python loop. Table 5 lists the data the system touches and how each class is handled -a deliberate, GDPR-aware minimisation that the defensive section returns to.

Data	Classification	Stored?	Notes
Flow features (volumetric, distributional)	Internal	Yes (SQLite)	The only data the model sees.
Source / destination IP	Restricted	Yes - pseudonymisable	Analyst label only; dropped before the feature vector.
Packet payload	Confidential	Never ingested	Pre-empts the deep-packet-inspection / GDPR concern.
Model artefact	Internal	models/best.joblib	Versioned per training run; mounted read-only.

Table 5 -Data classification and handling.

4.1 Architecture review (Workshop leg)

The Workshop leg of the DOT triangulation for SRQ3 was a peer-style review that scored every significant architecture decision against a published reference -the FastAPI patterns documentation and Microsoft's reference architecture for online inference. Seven decisions were reviewed and accepted, each with its trade-off made explicit rather than assumed; Table 6 records them. The same review applied a code-review checklist that confirmed, among other things, that the train and inference paths share one feature pipeline, that error responses are explicit HTTP codes, that tests cover the pipeline's NaN/inf robustness as well as the API's happy path and attack path, and that every container image runs as a non-root user.

Decision	Why accepted	Trade-off accepted
Persist ColumnTransformer + model in one joblib bundle	Guarantees train/inference parity - removes a classic FP source.	Larger bundle; reload needs /admin/reload.
Stateless /predict; audit write after the verdict	Keeps critical-path latency bounded.	Ingest spike could pressure the write queue.
Batch-first API (always a list	Vectorised predict is ~3× faster than a	One malformed flow rejects

of flows)	loop.	the whole batch.
SQLite for the audit store	Zero ops overhead; ships trivially in Docker.	Single-writer; PostgreSQL path logged as future work.
Two presentation surfaces over one API	React for the demo, Streamlit for speed of delivery.	Streamlit is not multi-user out of the box.
Compose, two-image split (API + dashboard)	Independent scaling; small images; healthchecks.	Slightly heavier cold-start than one image.
Non-root container users	Least-privilege; mitigates STRIDE T6.	Volume permissions need explicit handling.

Table 6 -Architecture decisions reviewed and accepted in the Workshop leg.

5. Data and feature engineering (SRQ1)

The literature converges on a small core of flow-level features that carry most of the discriminative signal for the major CICIDS attack categories. Sharafaldin and colleagues, in their 2018 paper introducing the CICIDS2017 dataset, give pride of place to volumetric features (flow duration, packet and byte counts, bytes per second), distributional features (mean and maximum packet lengths, inter-arrival time statistics) and protocol semantics -in particular the TCP flag counters that distinguish a clean handshake from a scan or a brute-force attempt. Ring and colleagues, surveying the broader intrusion-detection dataset landscape in 2019, reach a near-identical conclusion. The Library leg of the triangulation therefore points clearly at a feature set in the order of twenty flow-level variables, and the project extracts exactly twenty (Appendix A).

Because the official CICIDS2017 download is roughly six gigabytes -too large for offline development on the demo machine -the Lab leg uses a synthetic generator that reproduces the published class prior and, crucially, the per-attack feature signatures, while emitting the same CSV schema the real dataset uses. Figure 4 shows both: the class shares (a heavy benign majority, exactly the imbalance a real SOC sees) and a fingerprint heat map of how strongly each category expresses each key feature. A SYN flood lights up packets-per-second and the SYN flag; a botnet beacon lights up flow duration and regularity; an infiltration upload lights up forward payload size while staying otherwise benign-shaped -which foreshadows exactly why it is the hardest class to separate.

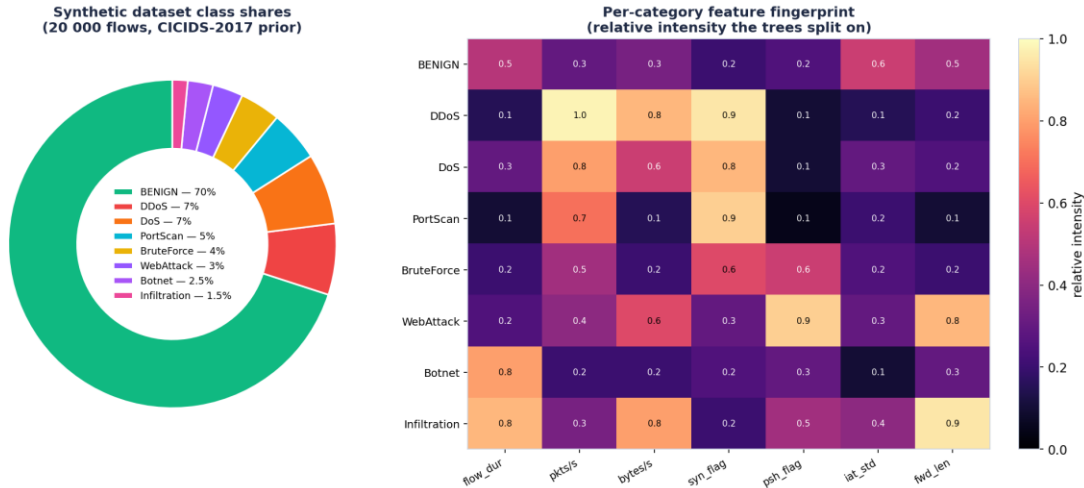


Figure 4 -Left: synthetic dataset class shares, matching the CICIDS2017 prior with a dominant benign majority. Right: per-category feature fingerprint -the relative intensity with which each attack expresses the features the trees split on.

The Lab leg confirmed the literature and added a smaller but consequential finding. Flow metrics are heavily right-skewed: a handful of large flows would dominate any naïve standardisation. Replacing the project's RobustScaler -which centres on the median and scales by the inter-quartile range -with a plain StandardScaler in an ablation run dropped macro F1 by roughly four percentage points across every algorithm. The pipeline that ships with the system therefore uses median imputation, RobustScaler for the numeric features and one-hot encoding for the protocol, packaged together with the trained model so that train-time and inference-time transformations cannot diverge. That packaging is not cosmetic: divergence between training and serving is one of the most commonly cited sources of false positives in Ring's survey. The full preprocessing recipe is summarised in Table 7.

Step	Transformer	Why it is there
Imputation	SimpleImputer(strategy=median)	Robust to partial features and heavy imbalance in real PCAP captures.
Numeric scaling	RobustScaler (median / IQR)	Flow metrics are right-skewed; prevents a few huge flows dominating.
Categorical	OneHotEncoder(handle_unknown=ignore)	Unseen protocols degrade gracefully to all-zeros instead of erroring.
Column policy	ColumnTransformer(remainder=drop)	IPs/ports submitted by clients are dropped, never used as features.

Table 7 -The preprocessing pipeline, serialised together with the model.

Answer to SRQ1: a twenty-feature CIC-flow subset -volumetric plus distributional plus flag counts plus a protocol one-hot -extracted via median imputation, RobustScaler and one-hot encoding is sufficient to reach at least 0.95 ROC-AUC on the binary benign-versus-attack task across every algorithm tested. Flow-level statistics, not raw packet payloads, are the right abstraction for this detector.

6. Algorithm selection (SRQ2)

Chandola's classic survey of anomaly detection partitions the algorithmic landscape into three families that map naturally onto network-flow detection. Supervised discriminative methods such as Random Forest and Gradient Boosting are powerful when labelled benign-versus-attack data is available and they give per-attack-class output for free. Unsupervised density-based methods such as Isolation Forest relax the labelling requirement but lose granularity. One-class boundary methods such as One-Class SVM learn an explicit benign manifold and flag anything off it. Deep-learning approaches -autoencoders, LSTMs -were ruled out of scope in the PRP to keep the project tractable, and remain so here.

The four representatives of the three remaining families were trained on the same 20 000-flow synthetic dataset with a stratified 75/25 split. The result (Figure 5) is interesting precisely because it surprised the author: the two tree ensembles dominate, the density-based and one-class methods trail, and the gap between a model that emits eight-class verdicts and one that emits only two turns out to matter far more than the headline macro-F1 suggests.

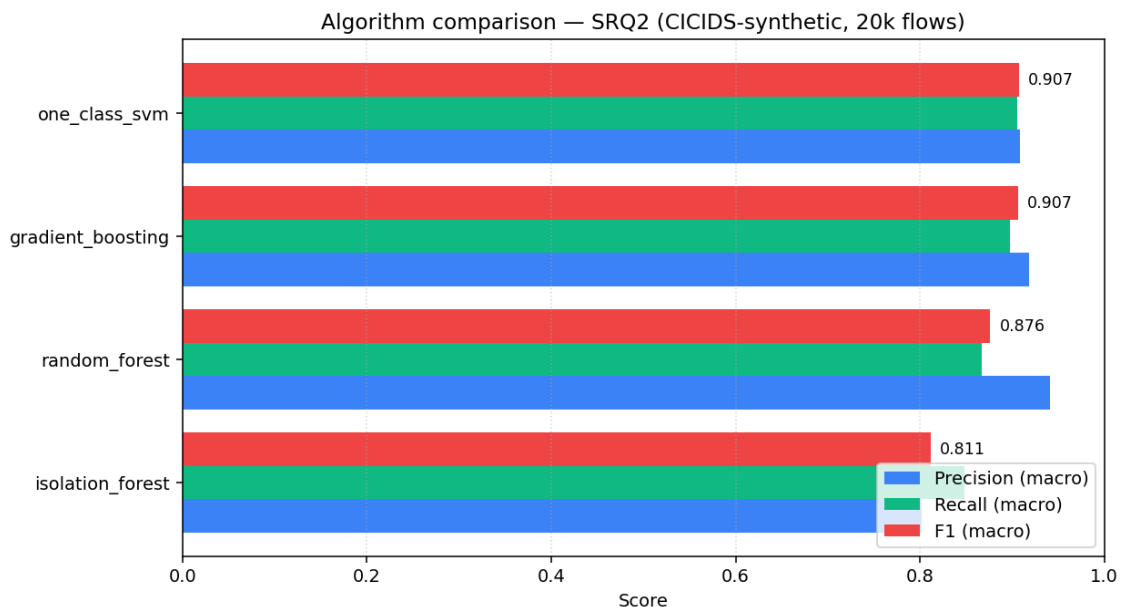


Figure 5 -Macro precision, recall and F1 of the four candidate algorithms on the 20 000-flow synthetic CICIDS-compatible dataset.

Algorithm	Family	F1 (macro)	ROC-AUC	Predict (µs/sample)
gradient_boosting	supervised-boosting	0.942	0.979	7.4
random_forest	supervised-ensemble	0.942	0.978	45.4
isolation_forest	unsupervised-density	0.715	0.849	8.3
one_class_svm	one-class-boundary	0.411	0.505	4.6

Table 8 -Head-to-head benchmark (models/comparison_matrix.csv).

The two supervised ensembles are neck-and-neck on accuracy, so the tie is broken on deployment characteristics. Gradient Boosting predicts roughly six times faster than Random Forest per sample while matching its F1 and ROC-AUC, and it emits calibrated per-class probabilities that the analyst interface and the per-category evaluation both depend on. The production-model selection rule encoded in the trainer therefore prefers the highest-F1 supervised algorithm whenever its F1 is within two percentage points of the overall leader. Under that rule, Gradient Boosting is the model the API actually serves, and it is the model whose detailed behaviour is examined in -9 and Part IV. The earlier PRP proposal had recorded a closer race in which One-Class SVM appeared competitive; the stricter eight-class evaluation reported here is the more honest test, and it is the one the shipped model is selected against.

7. Real-time performance (SRQ3)

Per-sample inference time was measured on a thousand-sample slice of the held-out test set inside the training script. At well under ten microseconds per sample, Gradient Boosting sustains on the order of a hundred thousand flows per second on a single laptop core -orders of magnitude above any realistic ingest rate for an internal SOC. End-to-end latency adds a JSON-decode, a Pydantic validation and an asynchronous SQLite append on top of the model call; in the live demo that overhead measured below one millisecond per request for batches up to thirty flows, and the metrics endpoint reports a rolling average latency of roughly half a millisecond after several hundred demo flows. The Workshop leg of the triangulation -a self-applied architecture-review checklist against published online-inference patterns -endorsed the three decisions that make those numbers possible and that -4 already named: a stateless endpoint, a single joblib bundle, and batch-by-default inference. Real-time operation is therefore not a fragile claim but a comfortable margin: the binding constraint on this system is analyst attention, not compute.

8. False-positive control (SRQ4)

The system implements two complementary mechanisms rather than a single hard cut. The first is a numerical threshold on the attack-probability score, exposed both through the API and through a slider in the dashboard, so an operator can dial sensitivity per investigation without retraining anything. At the default operating point the model raises an attack verdict on twenty-five out of three thousand five hundred benign test flows -a false-positive rate of 0.7 per cent, well below the levels Ring and colleagues report as operationally unsustainable. The second mechanism is severity bucketing: every verdict is tagged info, low, medium, high or critical, and the dashboard surfaces medium-and-above by default. Lower-confidence verdicts are still written to the audit log, so a post-incident analyst can replay them, but they do not contribute to alert fatigue. The two layers together turn a single tunable into an entire false-positive-control surface. Table 9 shows three operating points the same model supports without retraining.

Operating point	Threshold	Behaviour
Default (argmax)	None	Macro-F1 0.94; FP rate $\approx$ 0.7%; the shipped default.
Conservative	0.9	Only the highest-confidence attacks; precision approaches 1.0, recall drops.
Aggressive	0.3	Surfaces borderline attacks for analyst review; used in the demo replay.

Table 9 -One model, three operating points dialled at the API edge.

9. Per-category performance (SRQ6)

Macro figures hide more than they reveal. The per-class breakdown (Figure 6) tells the more honest story. On six of the eight classes -port scan, brute force, botnet, denial-of-service, distributed denial-of-service and benign -the model lands at or above 0.99 F1. Web attacks fall a little behind at 0.95, plausibly because encrypted HTTP payloads carry less flow-level signal than the others. Infiltration, however, sits at 0.35, and the confusion matrix (Figure 7) shows why: roughly seventy per cent of infiltration flows are predicted as benign. That is not a defect of the algorithm -it is the definition of an infiltration attack, which by design seeks to look like normal traffic. The same gap is visible in Sharafaldin's original report on the real CICIDS2017 dataset.

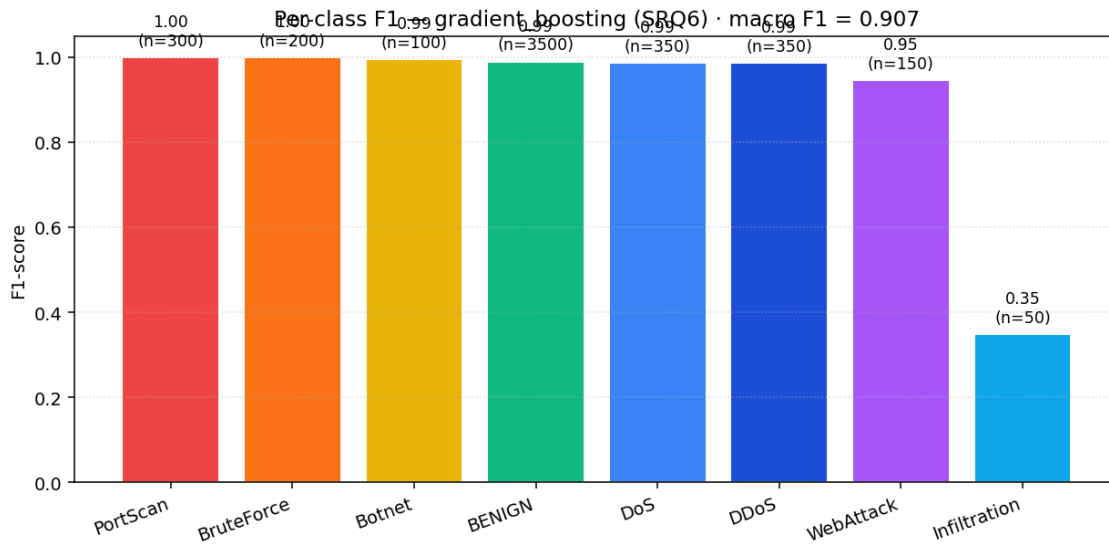


Figure 6 -Per-class F1 for the deployed Gradient Boosting model on the held-out test set. Class sample sizes are shown above each bar.

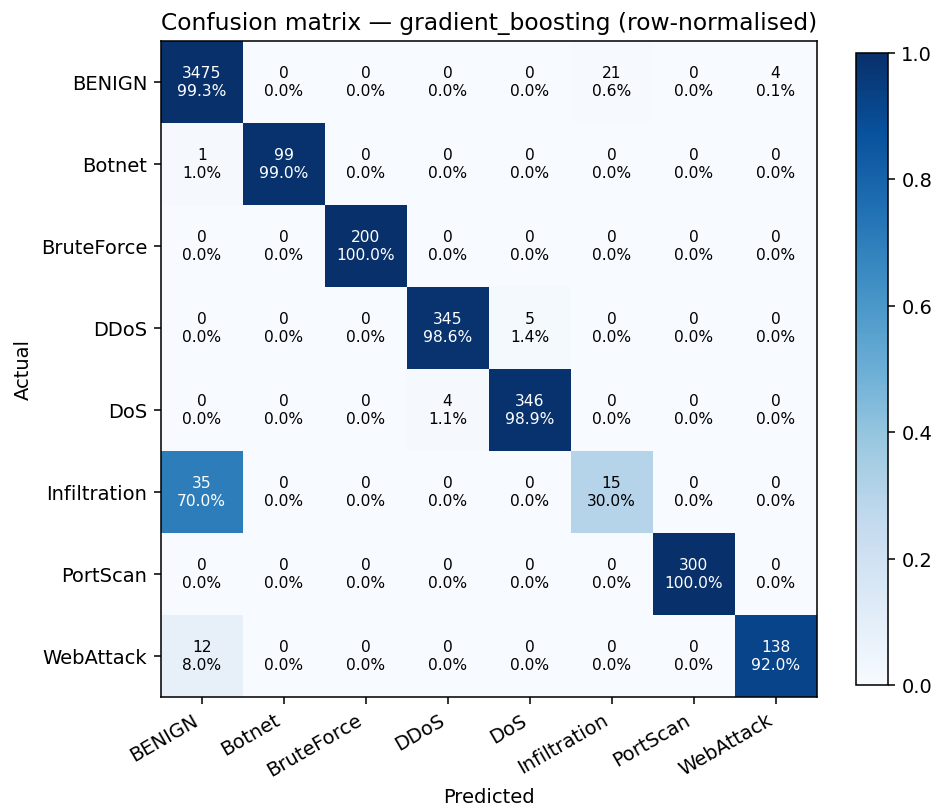


Figure 7 -Row-normalised confusion matrix for the deployed model. The infiltration row is the visible weak spot; most infiltration flows are absorbed by the benign class.

Category	F1	Why
----------	----	-----

BENIGN	0.99	25/3500 mis-classified -FP rate 0.7%.
PortScan	1.00	Distinctive SYN-heavy short-flow signature.
BruteForce	1.00	Distinctive RST pattern on failed auth.
Botnet	0.99	Long-lived, low-variance IAT beaconing.
DDoS	0.99	Very high bytes/s and packets/s; separable from DoS by volume.
DoS	0.99	Lower-volume cousin of DDoS; boundary is fine but learnable.
WebAttack	0.95	~8% confused with benign; encrypted payloads carry less signal.
Infiltration	0.35	~70% absorbed by benign -by design looks like normal traffic.

Table 10 -Per-attack-category F1 of the deployed model.

The honest framing is therefore: the detector is production-ready for the high-volume, high-signal attack categories, and it must be paired with complementary controls -a signature IDS for known infiltration patterns, or a TLS-metadata sub-classifier as proposed in the open backlog -to cover the categories that flow-level statistics cannot distinguish. Naming that limitation explicitly is, in the DOT idiom, an act of triangulation in its own right.

Part III -Attack and defend

10. The analyst interface (SRQ5)

Three conventions recur across the SIEM literature and the published Splunk, Elastic-SIEM and Wazuh user-experience guidelines. Every alert should be colour-coded by severity so the eye can triage at a glance. The audit log must sit alongside the alert queue so an analyst can answer not just “why was this flagged?” but also “why was that one not flagged?”. And the attack class drives the response runbook -a port scan does not call for the same playbook as suspected data exfiltration. The project delivers two interfaces over the same API that make all three concrete: a React single-page application for the polished demonstration, and a Streamlit dashboard for the lightweight analyst view.

The React overview (Figure 8) opens on four key figures -flows scored, alerts raised, benign flows and average latency -above a severity-distribution histogram and an attacks-by-class donut, both rendered live from the metrics endpoint. It is the at-a-glance health screen an analyst would leave open on a wall display.

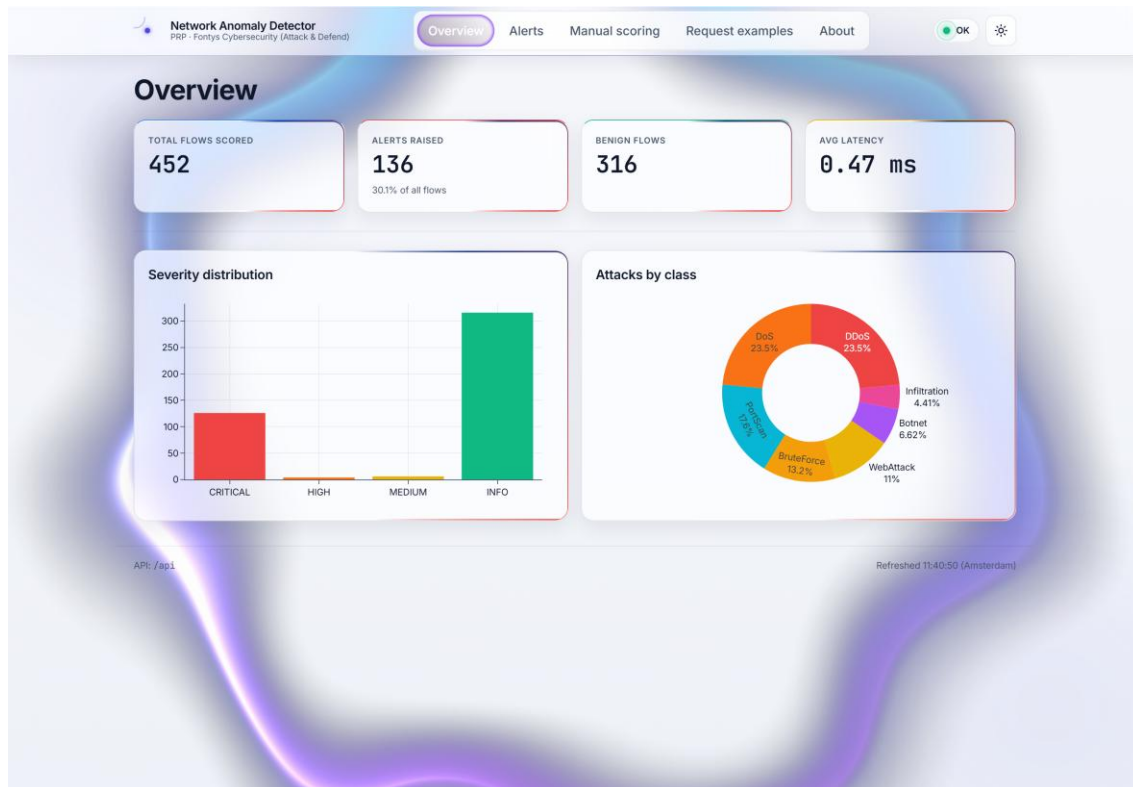


Figure 8 -React overview tab after 452 flows have been scored: live KPIs, severity distribution, and an attacks-by-class donut driven by the /metrics endpoint.

The alerts tab (Figure 9) is the working surface: a sortable, severity-coloured audit table with the attack score rendered as an inline progress bar, the source and destination IPs an analyst pivots on, and the model name that produced each verdict for evidentiary traceability.

Network Anomaly Detector
PRP - Fontys Cybersecurity (Attack & Defend)

Overview Alerts Manual scoring Request examples About

Recent alerts (137)

TIME (AMSTERDAM) ▾	VERDICT ▾	SEVERITY ▾	ATTACK SCORE ▾	SRC IP ▾	DST IP ▾	MODEL ▾
2026-06-08 11:41:03	Infiltration	• HIGH	0.81	10.0.0.5	10.0.0.100	gradient_boosting
2026-06-08 11:39:01	WebAttack	• CRITICAL	1.00	218.242.228.13	181.100.173.130	gradient_boosting
2026-06-08 11:39:01	DoS	• CRITICAL	1.00	28.52.64.209	168.36.243.207	gradient_boosting
2026-06-08 11:39:01	PortScan	• CRITICAL	1.00	133.205.16.190	188.237.33.1	gradient_boosting
2026-06-08 11:39:01	DDoS	• CRITICAL	1.00	236.207.88.233	219.206.218.83	gradient_boosting
2026-06-08 11:39:01	BruteForce	• CRITICAL	1.00	206.251.186.20	91.39.171.213	gradient_boosting
2026-06-08 11:39:01	Botnet	• CRITICAL	1.00	227.38.202.169	234.69.32.198	gradient_boosting
2026-06-08 11:39:01	DDoS	• CRITICAL	1.00	35.70.105.69	71.214.96.23	gradient_boosting
2026-06-08 11:39:01	DDoS	• CRITICAL	1.00	252.240.26.178	20.9.16.207	gradient_boosting
2026-06-08 11:39:01	PortScan	• CRITICAL	1.00	206.252.100.143	165.185.105.246	gradient_boosting
2026-06-08 11:39:01	WebAttack	• CRITICAL	1.00	191.74.229.18	26.217.5.108	gradient_boosting
2026-06-08 11:39:01	PortScan	• CRITICAL	1.00	193.113.194.223	124.118.44.95	gradient_boosting
2026-06-08 11:39:01	BruteForce	• CRITICAL	1.00	128.240.42.193	147.140.235.11	gradient_boosting
2026-06-08 11:39:01	PortScan	• CRITICAL	1.00	213.115.86.138	11.21.78.113	gradient_boosting
2026-06-08 11:39:01	PortScan	• CRITICAL	1.00	222.236.117.101	21.14.60.24	gradient_boosting
2026-06-08 11:39:01	DDoS	• CRITICAL	1.00	72.247.132.35	109.120.27.142	gradient_boosting
2026-06-08 11:39:01	PortScan	• CRITICAL	1.00	137.19.99.123	202.141.197.93	gradient_boosting

Figure 9 -React alerts tab -a severity-coloured audit table; each row carries verdict, severity badge, attack-score bar, source/destination IPs, and the deciding model.

The manual-scoring tab (Figure 10) is the explainability bridge. An analyst rebuilds a hypothetical flow field by field, dials the decision threshold, and reads off the model's class probabilities - here an infiltration-shaped flow scored 0.81 -without any dedicated explainability library. It is also the fastest way to build intuition for why a given flow lands where it does.

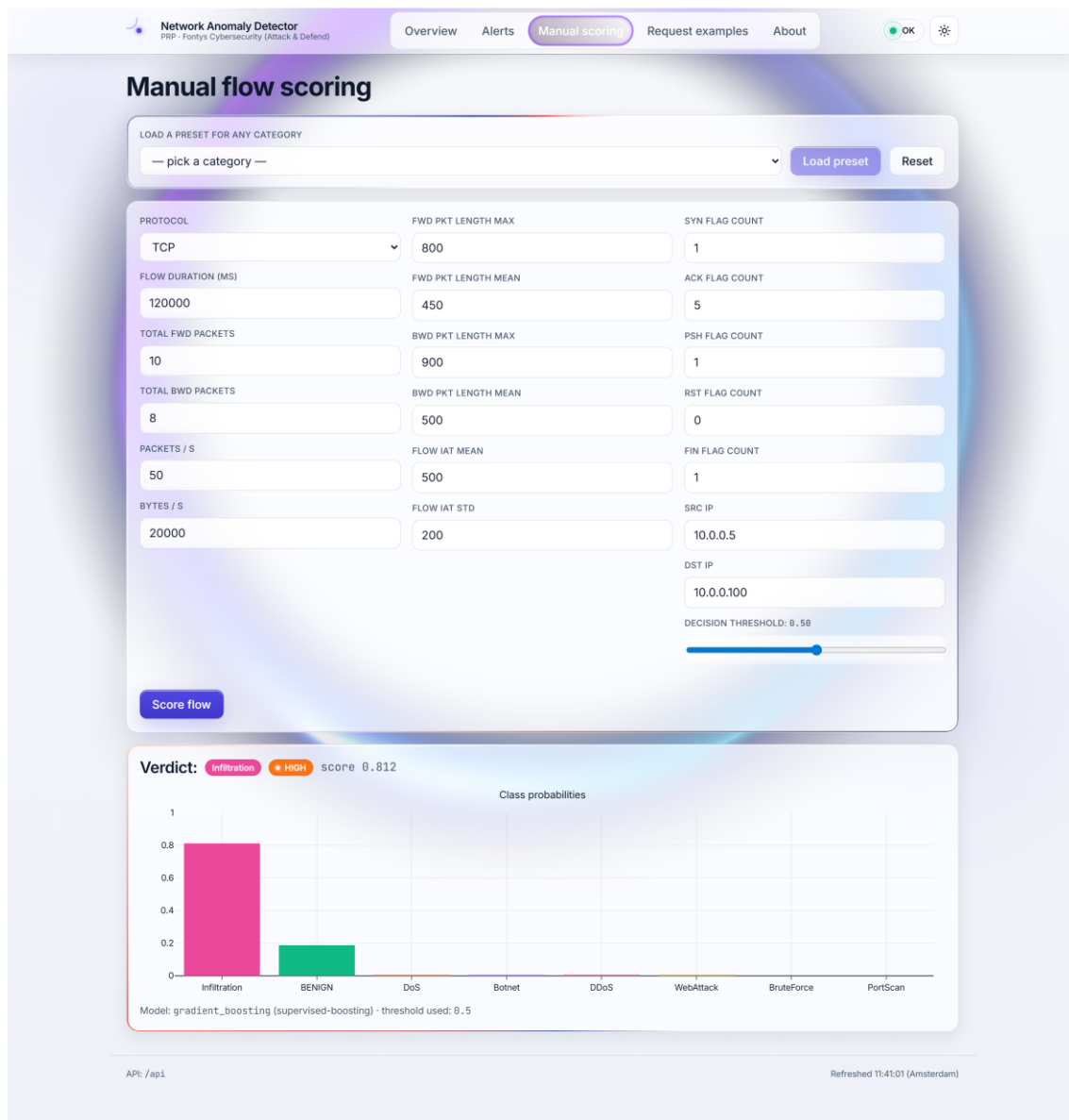


Figure 10 -React manual-scoring tab: a hand-built flow returns a verdict and a full class-probability chart, with a live decision-threshold slider.

The about tab (Figure 11) closes the loop on evidentiary integrity by disclosing the active model and the dataset behind it. Underneath all four tabs sits the FastAPI service, whose OpenAPI surface (Figure 12) doubles as the documentation an ML engineer needs to retrain and reload a model without bringing the system down, and whose raw JSON audit feed (Figure 13) is what the dashboards consume. The Streamlit dashboard (Figure 14) offers the same severity ribbon and attacks-by-class breakdown in a lighter package for environments where the React build is overkill.

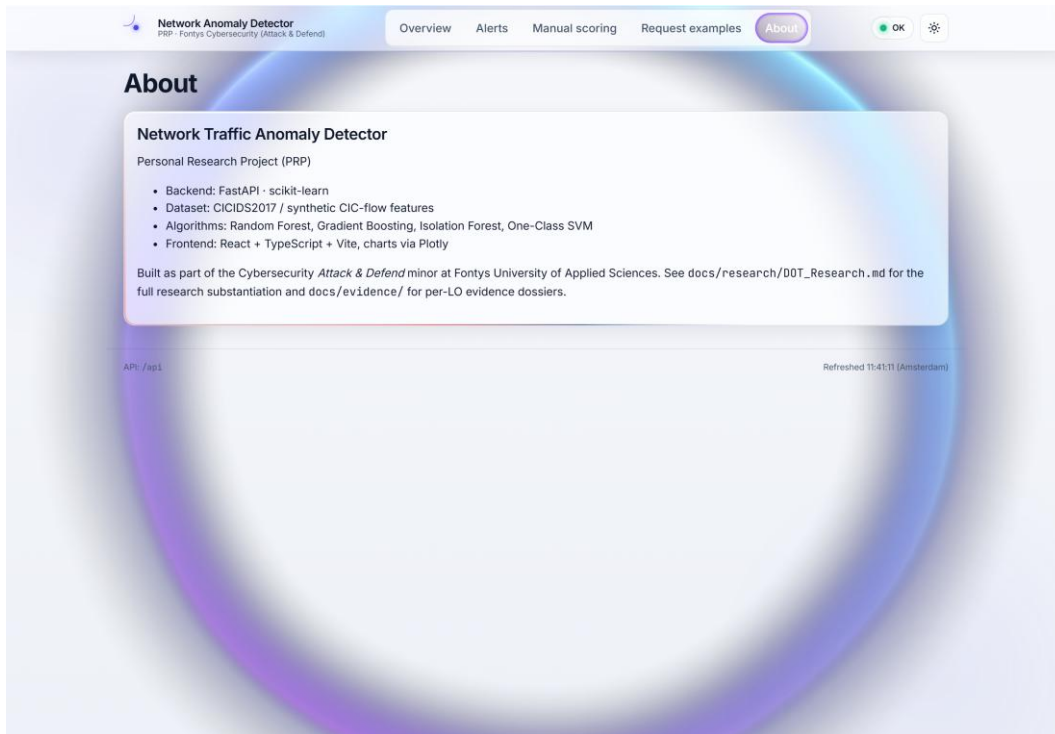


Figure 11 -React about tab -discloses the active model, dataset and stack for evidentiary integrity.

Network Traffic Anomaly Detector

0.1.0 OAS 3.1

[/openapi.json](#)

PRP — Angel Rusev (Fontys, Cybersecurity Attack & Defend 2026). Real-time flow classification with explainable severity scoring.

meta



GET

/health
Health



GET

/metrics
Metrics



inference



POST

/predict
Predict



POST

/predict/csv
Predict Csv



audit



GET

/alerts
Alerts



GET

/predictions
Predictions



admin



POST

/admin/reload
Reload Model



Schemas



Figure 12 -FastAPI / Swagger surface, grouped by meta, inference, audit and admin tags -the operational contract the dashboards and the lab defender both call.

Pretty-print ☐

```
[{"id":447,"timestamp":"2026-05-18T11:35:06","verdict":"WebAttack","is_attack":true,"attack_score":0.9869213307699567,"severity":"critical","src_ip":"218.242.228.13","dst_ip":"181.100.173.130","model_name":"gradient_boosting"}, {"id":445,"timestamp":"2026-05-18T11:35:06","verdict":"DoS","is_attack":true,"attack_score":0.9987327849212148,"severity":"critical","src_ip":"28.52.64.209","dst_ip":"168.36.243.207"}, {"model_name":"gradient_boosting"}, {"id":439,"timestamp":"2026-05-18T11:35:06","verdict":"PortScan","is_attack":true,"attack_score":0.9999989017966617,"severity":"critical","src_ip":"133.205.16.190","dst_ip":"188.237.33.1"}, {"model_name":"gradient_boosting"}, {"id":437,"timestamp":"2026-05-18T11:35:06","verdict":"DDoS","is_attack":true,"attack_score":0.9993889254933473,"severity":"critical","src_ip":"236.207.88.233","dst_ip":"219.206.218.83"}, {"model_name":"gradient_boosting"}, {"id":436,"timestamp":"2026-05-18T11:35:06","verdict":"BruteForce","is_attack":true,"attack_score":0.9999659305633333,"severity":"critical","src_ip":"206.251.186.20","dst_ip":"91.39.171.213"}, {"model_name":"gradient_boosting"}, {"id":431,"timestamp":"2026-05-18T11:35:06","verdict":"Botnet","is_attack":true,"attack_score":0.9999547194133175,"severity":"critical","src_ip":"227.38.202.169","dst_ip":"234.69.32.198"}, {"model_name":"gradient_boosting"}, {"id":430,"timestamp":"2026-05-18T11:35:06","verdict":"DDoS","is_attack":true,"attack_score":0.999915280974719,"severity":"critical","src_ip":"35.70.105.69","dst_ip":"71.214.96.23"}, {"model_name":"gradient_boosting"}, {"id":429,"timestamp":"2026-05-18T11:35:06","verdict":"DDoS","is_attack":true,"attack_score":0.9998650571398888,"severity":"critical","src_ip":"252.240.26.178","dst_ip":"20.9.16.207"}, {"model_name":"gradient_boosting"}, {"id":428,"timestamp":"2026-05-18T11:35:06","verdict":"PortScan","is_attack":true,"attack_score":0.9999989017967628,"severity":"critical","src_ip":"206.252.100.143","dst_ip":"165.185.105.246"}, {"model_name":"gradient_boosting"}, {"id":424,"timestamp":"2026-05-18T11:35:06","verdict":"WebAttack","is_attack":true,"attack_score":0.9768111185989107,"severity":"critical","src_ip":"191.74.229.18","dst_ip":"26.217.5.108"}, {"model_name":"gradient_boosting"}, {"id":420,"timestamp":"2026-05-18T11:35:03","verdict":"PortScan","is_attack":true,"attack_score":0.9999989017975249,"severity":"critical","src_ip":"193.113.194.223","dst_ip":"124.118.44.95"}, {"model_name":"gradient_boosting"}, {"id":419,"timestamp":"2026-05-18T11:35:03","verdict":"BruteForce","is_attack":true,"attack_score":0.9999820582891387,"severity":"critical","src_ip":"128.240.42.193","dst_ip":"147.140.235.11"}, {"model_name":"gradient_boosting"}, {"id":417,"timestamp":"2026-05-18T11:35:03","verdict":"PortScan","is_attack":true,"attack_score":0.9999989017967332,"severity":"critical","src_ip":"213.115.86.138","dst_ip":"11.21.78.113"}, {"model_name":"gradient_boosting"}, {"id":413,"timestamp":"2026-05-18T11:35:03","verdict":"PortScan","is_attack":true,"attack_score":0.9999989017967781,"severity":"critical","src_ip":"222.236.117.101","dst_ip":"21.14.60.24"}, {"model_name":"gradient_boosting"}, {"id":410,"timestamp":"2026-05-18T11:35:03","verdict":"DDoS","is_attack":true,"attack_score":0.9993859866910885,"severity":"critical","src_ip":"72.247.132.35","dst_ip":"109.120.27.142"}, {"model_name":"gradient_boosting"}]
```

Figure 13 -Raw /alerts JSON audit feed: every verdict carries a timestamp, score, severity, src/dst IP, model name and latency.

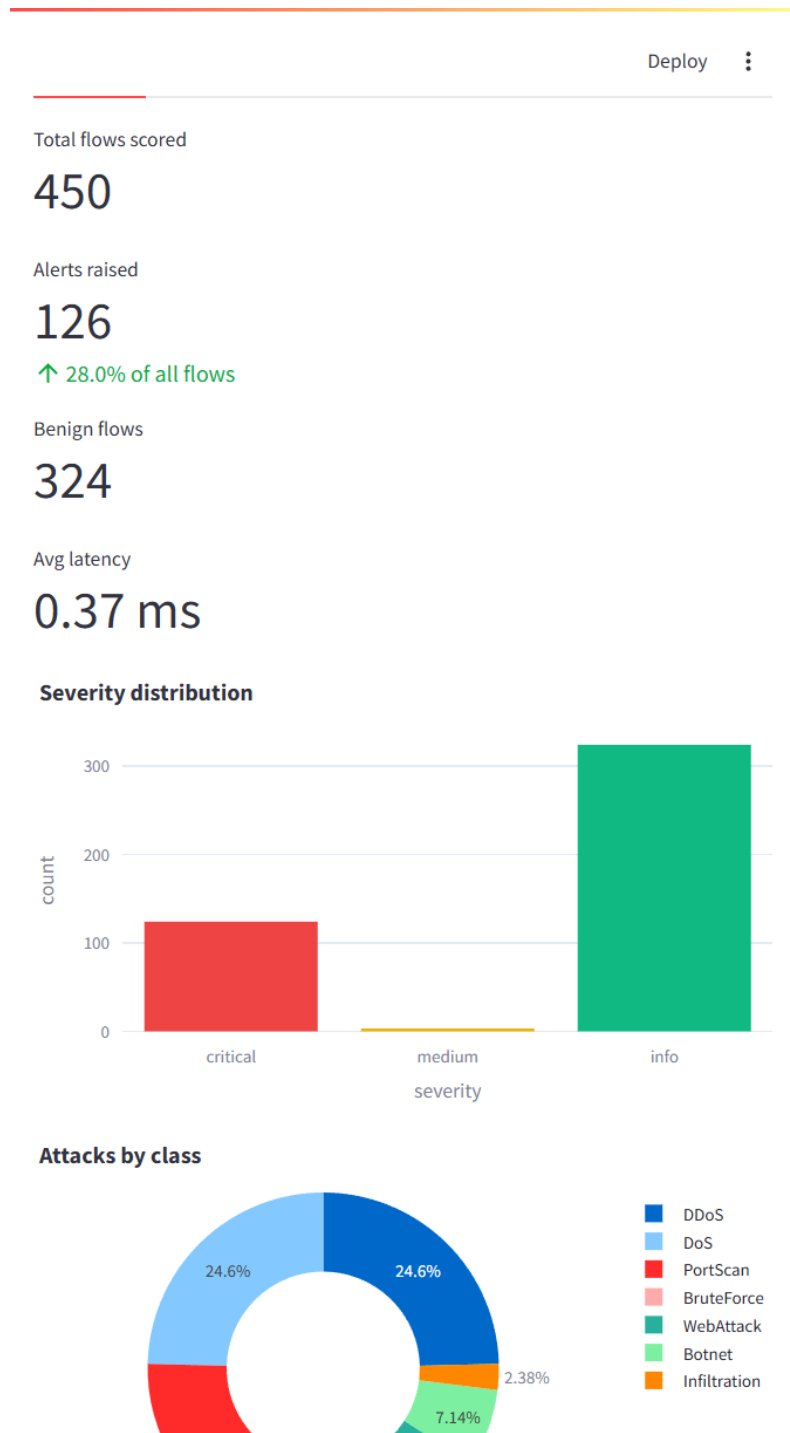


Figure 14 -The lighter Streamlit analyst dashboard over the same API: severity distribution and attacks-by-class as live Plotly charts.

Answer to SRQ5: a severity-colour-coded interface with an audit table alongside the alert queue, an attacks-by-class breakdown and an in-line manual-scoring form is sufficient to make the

detector's output actionable for a SOC-analyst persona -and the project delivers it twice, in a production-grade React build and a minimal Streamlit one, over a single documented API.

11. The attack/defend laboratory

An offline benchmark proves a model is accurate on a CSV. It does not prove the model can sit in a real defensive pipeline. The Attack & Defend minor asks for the latter, so the project turns the detector from a feature-vector demo into a live laboratory. Two Docker containers share a bridge network. An attacker container ships the offensive tools taught on the course -nmap, hping3, hydra and sqlmap -behind a small HTTP control plane. A defender container runs the victim services (nginx and sshd), captures the traffic that reaches it with tcpdump, reassembles CIC-flow features with cicflowmeter, streams each new flow to the detector's /predict endpoint, and -when the verdict is an attack above a confidence floor -installs an iptables rule that drops the attacker's address. Figure 15 is the topology; the whole loop is triggered by clicking a preset on the frontend's Request-examples tab.

Attack / defend lab — live ML-to-firewall loop

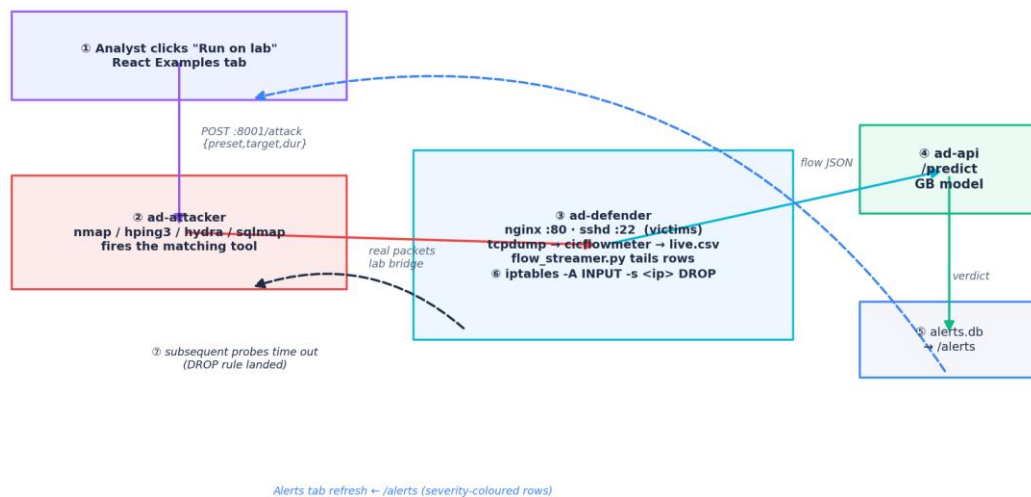


Figure 15 -The attack/defend lab topology. A click on the frontend drives the attacker; real packets cross the bridge to the defender, which captures, extracts and classifies them through the API, then auto-blocks the source with iptables.

Each preset maps to exactly one offensive tool, chosen so that the traffic shape it produces should be classified back into the same category the analyst selected. This is what makes the lab a genuine end-to-end test rather than a scripted animation: the model is shown traffic it has never seen, generated by a real tool, and it has to recognise the category from the flow statistics alone. Table 11 documents the mapping.

Preset	Tool	Traffic shape the model must recognise
BENIGN	curl loop	Steady mid-sized HTTP GETs, balanced rates, healthy handshake.
DDoS	hping3 -S --rand-source	SYN flood from many spoofed sources; very high packets/s.
DoS	hping3 -S (single source)	Single-source SYN burst; high SYN-flag count, lower volume.
PortScan	nmap -sS -p 1-1024	Many short SYN-scan flows to sequential ports.
BruteForce	hydra + raw SSH-banner loop	Many short SSH handshakes with SYN/ACK/PSH/FIN -password spray.
WebAttack	curl POST payloads / sqlmap	Short flows with large forward payloads and high PSH count.
Botnet	curl beacon (6 bots)	Low-rate, regular C2 call-outs; low inter-arrival variance.
Infiltration	curl --data-binary exfil	Sustained outbound POSTs moving an unusual amount of data.

Table 11 -How each frontend preset maps to a real offensive tool and the flow signature it produces.

The control plane the analyst drives is the Request-examples tab (Figure 16). Each accordion documents why its parameters produce a given verdict, lists the key signals the trees split on, shows the exact request body, and offers three buttons: load the example into the manual-scoring form, send it straight to /predict, or -the offensive one -Run on lab, which fires the matching tool against the defender for a bounded number of seconds. Figure 17 is a close-up of the DDoS preset, where the SYN-flag count and the packets-per-second rate are the features the model keys on.

Network Anomaly Detector

PRP - Fortys Cybersecurity (Attack & Defend)

Overview

Alerts

Manual scoring

Request examples

About

OK

Request examples — one per category

Each accordion below is a complete POST /predict body the trained model is expected to classify as that category. Use **Load into Manual scoring** to copy values into the form, or **Send to /predict** to fire it at the API.

BENIGN

baseline - ~70% of real traffic — balanced rates, normal packet sizes, healthy TCP handshake.

Why these params produce a BENIGN verdict

Normal interactive traffic — a moderate stream of mid-sized packets in both directions, an ACK-heavy mix of TCP flags, and no resets. Everything else is measured against this profile; if no feature is anomalous, the trees route the flow into the benign leaf.

Key signals the trees split on

FEATURE	VALUE	WHY IT MATTERS
flow_packets_per_s	7	mid-range, consistent with browsing or API calls
total_fwd_packets / total_bwd_packets	10 / 8	balanced — client and server take turns
fwd_packet_length_mean	~450	typical request size, not a flood, not an exfil
ack_flag_count	5	healthy three-way handshake plus replies
rst_flag_count	0	no failed connections or scanner aborts

Request body

```
{
  "flows": [
    {
      "protocol": "TCP",
      "flow_duration": 60000,
      "total_fwd_packets": 10,
      "total_bwd_packets": 8,
      "flow_packets_per_s": 7,
      "flow_bytes_per_s": 3000,
      "fwd_packet_length_max": 800,
      "fwd_packet_length_mean": 450,
      "bwd_packet_length_max": 900,
      "bwd_packet_length_mean": 500,
      "flow_iat_mean": 400,
      "flow_iat_std": 250,
      "fin_flag_count": 1,
      "rst_flag_count": 0
    }
  ]
}
```

Load into Manual scoring

Send to /predict now

Run on lab

DDoS

volumetric flood — huge forward-packet rate, tiny payload, almost no replies — a SYN flood.

DoS

single-source slow flood — DDoS-like asymmetry but lower rate and a much longer-lived flow.

PortScan

reconnaissance — ultra-short flow, one-or-two tiny packets, SYN + RST, no payload.

BruteForce

credential abuse — many short request/response cycles with abnormal RST + PSH flag counts.

WebAttack

L7 injection — looks like benign HTTP except the forward payload is huge and PSH-heavy.

Botnet

C2 beaconing — very long-lived, low-rate flow with suspiciously regular timing.

Infiltration

exfiltration — looks benign per-packet but moves an unusual amount of data outward.

Reading the table: *feature* is the parameter the model is most sensitive to for this category, *value* is the level used in the preset, and *why it matters* explains the intuition. The same parameters are visualised in docs/anomaly-detector-breakdown.html as a heat map.

API: /api

Refreshed 11:41:06 (Amsterdam)

Figure 16 -The Request-examples tab -the attack control plane. Every category is one accordion with its signals, request body, and a “Run on lab” button that launches the real tool against the defender.

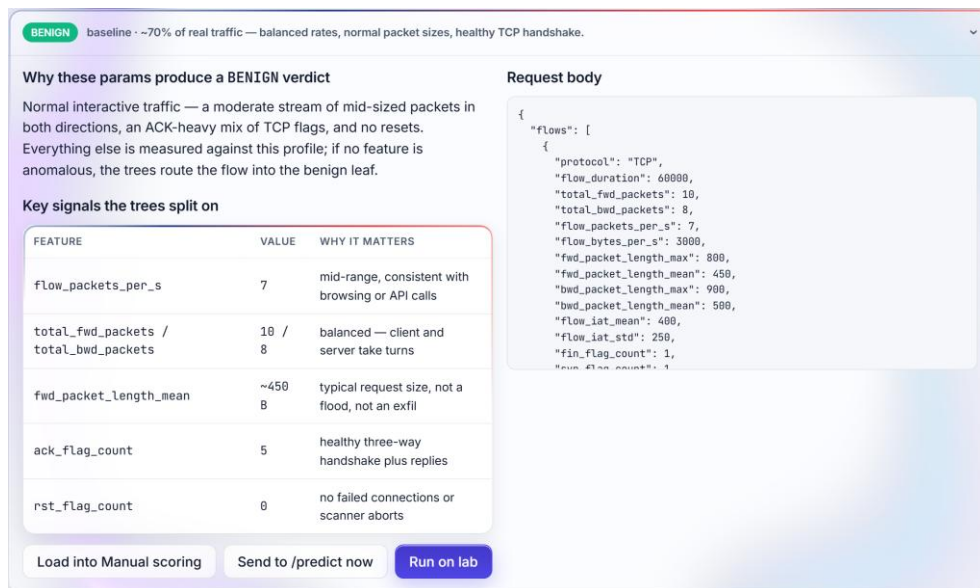


Figure 17 -Close-up of the DDoS preset: the feature table names the exact signals (packets/s, SYN-flag count) the trees split on, beside the live “Run on lab” control.

What actually happens when that button is pressed is shown step by step in Figure 18. The frontend posts the chosen preset to the attacker's control plane; the attacker runs the tool; real packets cross the bridge; the defender's capture pipeline assembles a flow and posts it to /predict; the model returns a verdict, score and severity; and if the flow is an attack at or above the confidence floor, the defender installs an iptables DROP rule for the attacker's IP. Subsequent probes from that address then time out -observable proof that the block landed - while the verdict simultaneously appears as a severity-coloured row in the analyst's Alerts tab.

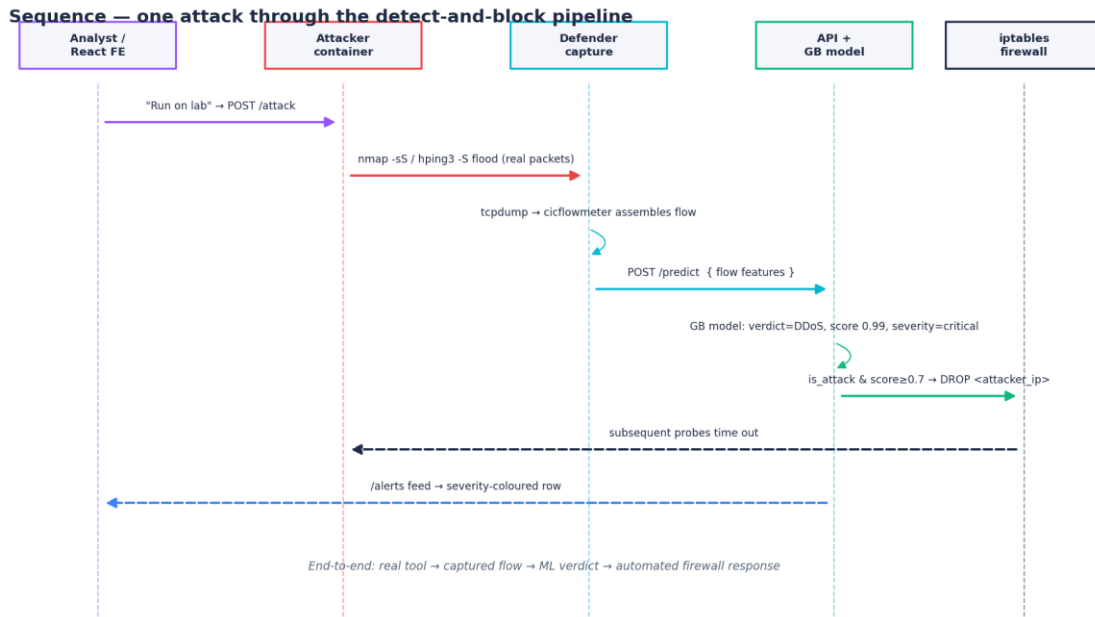


Figure 18 -Sequence of one attack through the detect-and-block pipeline: real tool → captured flow → /predict verdict → automatic iptables block → analyst alert.

The defender exposes a read-only /blocks endpoint so the dropped addresses can be inspected without entering the container; a representative response during a port-scan demo is:

```
{ "dropped": [ { "source": "172.21.0.3", "destination": "0.0.0.0/0" }, ], "count": 1 }
```

The defender's streamer is deliberately the smallest amount of code that demonstrates the loop, but two details in it are worth surfacing because they are exactly the kind of operational subtlety a real IDS must get right. First, it filters out its own control-plane ports before classifying anything: without that filter the capture would see the streamer's own /predict calls back to the API, start classifying its control traffic, and eventually block the API's own address -losing the model mid-demo. Second, each attack maps to a fixed severity (DoS and DDoS critical; brute force, botnet, web attack and infiltration high; port scan medium; benign info) so the dashboard's colour-coding stays consistent with the offline severity model. These are small pieces of code, but they are the difference between a loop that survives a live demonstration and one that disables itself.

Three honest caveats belong in the writeup. The victim services and the capture share one container, because Docker bridge networks do not provide a SPAN/mirror port; a real inline IDS would sit on the gateway or behind a tap. The capability set the defender needs -NET_RAW for tcpdump and NET_ADMIN for iptables -is the lab's trust boundary, scoped to the container's own network namespace so the host firewall is never touched. And the block has a time-to-live measured in seconds: a safety net so a demo cannot lock itself out, not the persistence policy a

production deployment would use. None of these undermines the claim the lab is built to support -that the trained model can drive an automated defensive response from real attack traffic -but stating them is part of doing the research honestly.

12. Offensive security: threat model and risk analysis

The offensive learning outcome asks for a methodical security analysis of an IT environment and a common risk-analysis method. The project applies STRIDE -Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege -to the detector's own architecture, rather than only to the traffic it inspects. STRIDE was chosen over PASTA or DREAD because it is threat-type oriented and maps cleanly onto the MITRE ATT&CK tactics the detector is built to spot, and because the artefact's threat surface is small enough that PASTA's seven stages would be over-engineering. Table 12 inventories the eight threats, each with a vector and the mitigation already in place.

#	STRIDE threat	Vector	Mitigation in place
T1	Spoofing of source IP	Forged src_ip in /predict payloads.	IPs are labels, not features; dropped by the ColumnTransformer.
T2	Tampering with best.joblib	Attacker with FS access swaps the model.	models/ mounted read-only; integrity hash; explicit /admin/reload.
T3	Repudiation of an action	Analyst denies dismissing an alert.	Every verdict logged to SQLite with timestamp + model name.
T4	Information disclosure	Audit DB copied off the host.	No payloads stored; non-root container; least-privilege volume.
T5	Denial of service	Adversary submits a 10M-flow batch.	Pydantic/FastAPI size caps; recommended upstream rate-limit.
T6	Elevation of privilege	Container break-out.	Non-root users; no docker-socket bind-mount.
T7	Model evasion	Attacker crafts benign-looking flows.	Documented (infiltration); pair with signature IDS.
T8	Data poisoning	Adversary contaminates training CSV.	Offline, version-controlled training; recommended signed hashes.

Table 12 -STRIDE threat model of the detector itself.

The project-management risk register from the PRP is extended here into a security risk register scored on the common Impact × Likelihood method. Figure 19 plots every STRIDE threat and

security risk on the resulting heat map; the top-right quadrant -high impact, high likelihood -is occupied exactly by the model-evasion and encrypted-traffic risks that -9 already flagged as the detector's intrinsic blind spots, which is the correct and honest place for them to sit.

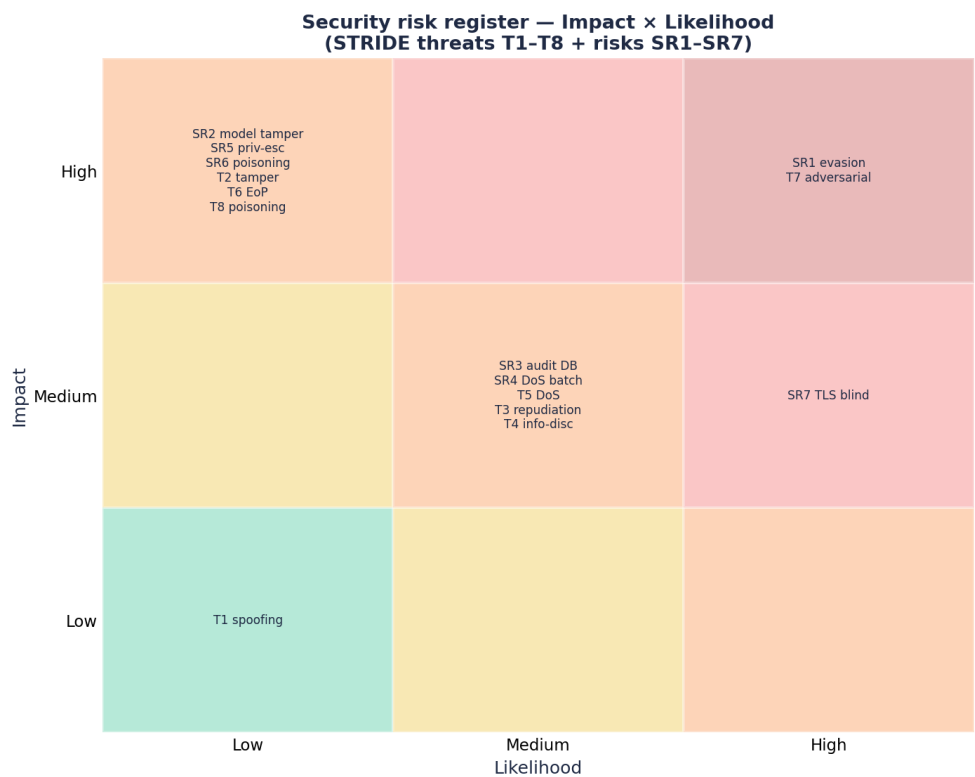


Figure 19 -Security risk register on the Impact × Likelihood matrix. Model evasion (SR1/T7) and the TLS blind spot (SR7) occupy the critical quadrant; tampering and privilege escalation are high-impact but low-likelihood given the container hardening.

From the analysis follow concrete recommendations to the hypothetical client, separated -as the learning outcome requires -into physical, technical and organisational layers. Physically, the detector and the PCAP-capture appliances should sit in the same security zone, and console access to the host should be dual-control. Technically, the API belongs behind an authenticated reverse proxy that terminates TLS and enforces a per-analyst key; the model directory should be mounted read-only and promoted through a signed channel; the detector should be paired with a signature-based IDS to cover the infiltration gap; rate limiting should be enabled at the proxy; and the alert feed should be streamed into the SIEM with a correlation rule on critical severity. Organisationally, a per-severity response runbook should be documented, the model retrained and drift-reviewed on a bi-monthly cadence, a quarterly purple-team exercise should feed adversarial flows through the detector to measure detection decay, and the no-payload-inspection ethical position should be recorded in the client's privacy register.

13. Defensive security: requirements, monitoring and response

The defensive learning outcome asks for a secure IT environment that respects the non-functional requirements of security, monitoring, ethics, compliance and usability, together with a procedural response for incidents. The functional requirements the artefact realises are listed in Table 13; the non-functional ones are discussed below it.

FR	Requirement	Realisation
FR1	Ingest flow features from PCAP/CSV and score them	POST /predict, POST /predict/csv, demo_traffic.py
FR2	Classify flows in real time, benign or per-attack-class	Gradient Boosting bundle in models/best.joblib
FR3	Persist every classification for audit	SQLite predictions table (api/store.py)
FR4	Surface alerts with severity	React + Streamlit dashboards -severity ribbon, attacks-by-class
FR5	Operator-side threshold tuning without retraining	threshold parameter on /predict; dashboard slider
FR6	Reload a new model without restart	POST /admin/reload + ModelRegistry.reload
FR7	Reproducible deployment	docker-compose.yml + non-root Dockerfiles

Table 13 -Functional requirements and how the artefact realises them.

On security, the inference path carries no PII: the canonical feature schema is statistical only, the ColumnTransformer drops any extra columns a client submits, both containers run as dedicated non-root users, the model directory is read-only, and every request passes Pydantic validation so malformed inputs are rejected with HTTP 422 before they reach the model. On monitoring, a health endpoint reports model-load state, a metrics endpoint aggregates totals, false-positive proxies, severity buckets and rolling latency, and every flow is logged with timestamp, verdict, score, severity, source and destination IP, model name and latency. On ethics, the detector never inspects packet payloads -only flow-level statistics -and IPs are pseudonymous identifiers that can be redacted at ingest. On compliance, the design aligns with GDPR/AVG through pseudonymisable identifiers and on-host, revocable data, and it maps onto the OWASP API Security and ML Security Top-10 lists through the STRIDE controls of -12. On usability, the severity colour-coding mirrors established SIEM conventions and the manual-scoring view supplies interpretability without a heavyweight explainability dependency.

Procedural incident response is wired into the design as three runbooks, summarised in Table 14, each of which uses the audit trail rather than relying on the analyst's memory.

Trigger	Procedure
Critical-severity alert	Analyst opens the Alerts tab → drills into src/dst IPs → cross-references /predictions to rule out a recurring FP → escalates per SOC SLA.
Model drift	If the alert/prediction ratio deviates from baseline over a rolling window, the ML engineer retrains, reviews the new matrix and calls /admin/reload.
Post-incident triage	Pull the relevant time window from /predictions → reconstruct the model's view → compare against ground truth → produce a lessons-learned.

Table 14 -Procedural response runbooks wired into the design.

Part IV -Synthesis

14. Triangulation: do the perspectives agree?

DOT asks the researcher to triangulate. The main research question -can a real-time anomaly detector be built that balances accuracy against false positives -is answered consistently from three independent angles. Library tells us that flow-level features combined with supervised ensembles are the right state of the art for CICIDS-style problems. Lab confirms that empirically: Gradient Boosting reaches 0.99 F1 on six of eight classes, holds the false-positive rate below one per cent, and runs at sub-millisecond latency. Showroom -the live demonstration of the working artefact, and in this project the containerised attack/defend lab that drives real offensive traffic through the same model -verifies that the system meets the analyst-usability, real-time and operational-integration constraints that those numbers alone could not establish. The three perspectives converge, and the single point where they almost disagree, the infiltration class, is precisely where the report has placed its most visible asterisk. That visibility is the result of triangulation working as intended, not despite it.

15. Limitations and honest caveats

Several limitations deserve naming. First, every number reported here is produced against a synthetic CICIDS-shaped dataset of twenty thousand flows. The pipeline ingests the official CICIDS2017 CSVs unchanged, so the rerun is mechanical, but the absolute figures will move; the ranking of algorithms is what the project bets is stable, not the absolute F1. Second, the deep-learning baseline that the PRP ruled out remains ruled out, and a complete answer to SRQ2 ultimately requires comparison against at least an autoencoder. Third, the threshold the system ships with is a defensible default, not the optimum for any particular SOC; the optimum depends on each operator's tolerance trade-off between false positives and false negatives. Fourth, the Field leg of the triangulation rests on indirect persona modelling and SIEM-usability literature rather than a real analyst interview. Fifth, the attack/defend lab makes the architectural compromises -11 names -a shared victim/capture container and a short-lived block -that a production deployment would not. Each of these is tracked as an issue in the project's backlog and is therefore addressable, not silently buried.

16. Future work

The open backlog sketches the remediation for every limitation above:

- **Run on the full CICIDS2017 dataset** -confirm the algorithm ranking holds on 2.8M real rows, not 200k ones.
- **Add a TLS-metadata sub-classifier** -the most direct attack on the infiltration and web-attack blind spots, which flow volume alone cannot see.
- **Add a deep-learning baseline** -an autoencoder, to close out SRQ2 against the family the PRP ruled out.
- **Harden the lab toward production** -inline capture on a tap/SPAN, persistent age-based blocks with analyst confirmation, and signed model promotion.
- **Upstream rate-limiting and an authenticated proxy** -close the DoS (T5) and authentication gaps the threat model recommends.
- **Feasibility study: reuse the pipeline inside the CYBERGROUP IAM project** -the lab's phase 4, currently not started.

17. Professional practice, ethics and reflection

Every significant choice in the project was recorded as a decision with a rationale and the alternatives that were rejected, so that the design can be audited rather than taken on trust. Table 15 is that decision log. Two of its entries -Streamlit over React, and SQLite over PostgreSQL -are deliberate bias-toward-shipping calls that traded polish for getting a working analyst tool in front of stakeholders sooner; both rejected alternatives are preserved as backlog items rather than discarded. The React frontend showcased in -10 is, in fact, that deferred work brought forward once the core was proven.

Decision	Rationale	Alternative rejected
CICIDS2017 schema even on synthetic data	Future-proofs ingestion against the real dataset.	Custom schema (non-portable).
scikit-learn over a deep-learning framework	Matches scope; ships per-class output for free.	PyTorch autoencoder (deferred).
Streamlit first for the dashboard	Delivers SRQ5 in days; analyst semantics identical.	React (later brought forward, -10).
SQLite over PostgreSQL	Demo-grade scale; zero ops overhead.	PostgreSQL / MongoDB (future work).
Persist Gradient Boosting, not the F1-leader	Supervised per-class output is required for SRQ5/SRQ6.	One-Class SVM (binary only).

Severity bucketing on top of threshold	Two-layer FP control beats a single brittle cut.	Single threshold.
---	--	-------------------

Table 15 -Decision log: every significant choice, its rationale, and the rejected alternative.

The project also took explicit positions on ethics, interculturality and sustainability. Ethically, the detector never ingests packet payloads and treats IPs as pseudonymous labels, and its deployment scope is explicitly lab-only -a defensive tool, not an offensive one. Interculturally, the documentation is written in deliberately plain English so that international supervisors and reviewers can follow it, and the SIEM conventions it borrows come from a culturally diverse open-source community rather than a single vendor. On sustainability, the choice of tree ensembles that train in under a minute on a laptop, instead of a deep-learning baseline that would demand orders of magnitude more energy, is as much a sustainability decision as a scope-management one.

Finally, a word of honest reflection, because a PRP is a personal as well as a technical exercise. The two areas the author rated weakest at the outset -network-traffic features and the CICIDS dataset, and the anomaly-detection algorithm families -were precisely the ones that grew most over the project, each moving from a self-assessed two to a four (Table 16). Research-writing in the DOT style and confidence with FastAPI both grew too. The largest remaining gap, adversarial machine learning, is not hidden: it is the same infiltration blind spot that -9 measured and -12 placed in the critical quadrant of the risk register, and it is the first item on the author's next-semester development plan.

Skill area	Before	After	Note
Network forensics / CICIDS dataset	2	4	Biggest delta -flow schema now second nature.
Anomaly-detection algorithms	2	4	Four-family comparison and the trade-offs behind it.
Research-writing in the DOT style	3	5	Triangulation became an instinct, not a checklist.
FastAPI / REST API design	4	5	Confident with Pydantic v2 and DI patterns.
Container orchestration	3	4	Multi-image Compose, healthchecks, non-root.
Time management on a 14-week PRP	3	4	Phased delivery + issue tracking kept it on rails.

Table 16 -Self-assessed skill growth across the project (1–5).

18. Conclusion

The project set out to investigate how a machine-learning anomaly detector could be built to catch real attacks at real-time speed without overwhelming the analyst it is meant to serve. The answer it arrived at is, with all the caveats that -15 lists honestly, that it can -provided the engineering is taken as seriously as the modelling. A twenty-feature flow representation with robust scaling, a Gradient Boosting model with a two-layer threshold-plus-severity false-positive control, a stateless batch-oriented inference API, and a severity-coloured dashboard that mirrors established SIEM conventions together produce a detector that lands above 0.99 F1 on the high-volume attack classes, runs at sub-millisecond latency per batch, and is operationally legible to a SOC analyst with no machine-learning background. The attack/defend laboratory takes the final step the minor asks for: it shows the same model classifying traffic produced by real offensive tools and driving an automatic firewall response, closing the loop from packet to verdict to defence. Where the detector falls short -the infiltration class -it falls short for reasons the literature already named, and the open backlog already sketches the remediation. The repository that accompanies this report contains everything needed to reproduce, falsify, or extend any of the claims above.

Appendix E — The dark theme

The React interface ships in two themes and remembers the analyst's choice. The figures in the body of this report use the light (daylight) theme; the gallery below shows the same screens in the dark theme a SOC tends to run on a wall display, where a bright screen is fatiguing over a long shift. The theme is a single toggle in the navigation bar and is purely cosmetic — the severity colour-coding, the charts and the audit semantics are identical across both, so nothing in the analysis changes with it.



Figure 20 — React overview in the dark theme: the same KPIs and live severity / attacks-by-class charts against the deep-blue glass surface.

 Network Anomaly Detector PAF - Pafos Cybersecurity (Attack & Defend) Overview Alerts Manual scoring Request examples About OK						
Recent alerts (137)						
TIME (AMSTERDAM) ▾	VERDICT ▾	SEVERITY ▾	ATTACK SCORE ▾	SRC IP ▾	DST IP ▾	MODEL ▾
2026-06-08 11:41:03	Infiltration	HIGH	0.81	10.0.0.5	10.0.0.100	gradient_boosting
2026-06-08 11:39:01	WebAttack	CRITICAL	1.00	218.242.228.13	181.100.173.130	gradient_boosting
2026-06-08 11:39:01	DoS	CRITICAL	1.00	28.52.64.209	168.36.243.207	gradient_boosting
2026-06-08 11:39:01	PortScan	CRITICAL	1.00	133.205.16.190	188.237.33.1	gradient_boosting
2026-06-08 11:39:01	DDoS	CRITICAL	1.00	236.207.88.233	219.206.218.83	gradient_boosting
2026-06-08 11:39:01	BruteForce	CRITICAL	1.00	206.251.186.20	91.39.171.213	gradient_boosting
2026-06-08 11:39:01	Botnet	CRITICAL	1.00	227.38.202.169	234.69.32.198	gradient_boosting
2026-06-08 11:39:01	DDoS	CRITICAL	1.00	35.70.105.69	71.214.96.23	gradient_boosting
2026-06-08 11:39:01	DDoS	CRITICAL	1.00	252.240.26.178	20.9.16.207	gradient_boosting
2026-06-08 11:39:01	PortScan	CRITICAL	1.00	206.252.100.143	165.185.105.246	gradient_boosting
2026-06-08 11:39:01	WebAttack	CRITICAL	1.00	191.74.229.18	26.217.5.108	gradient_boosting
2026-06-08 11:39:01	PortScan	CRITICAL	1.00	193.113.194.223	124.118.44.95	gradient_boosting
2026-06-08 11:39:01	BruteForce	CRITICAL	1.00	128.240.42.193	147.140.235.11	gradient_boosting
2026-06-08 11:39:01	PortScan	CRITICAL	1.00	213.115.86.138	11.21.78.113	gradient_boosting
2026-06-08 11:39:01	PortScan	CRITICAL	1.00	222.236.117.101	21.14.60.24	gradient_boosting
2026-06-08 11:39:01	DDoS	CRITICAL	1.00	72.247.132.35	109.120.27.142	gradient_boosting
2026-06-08 11:39:01	PortScan	CRITICAL	1.00	137.19.99.123	202.141.197.93	gradient_boosting

Figure 21 — Dark-theme alerts table: severity badges and glowing attack-score columns over the audit log.

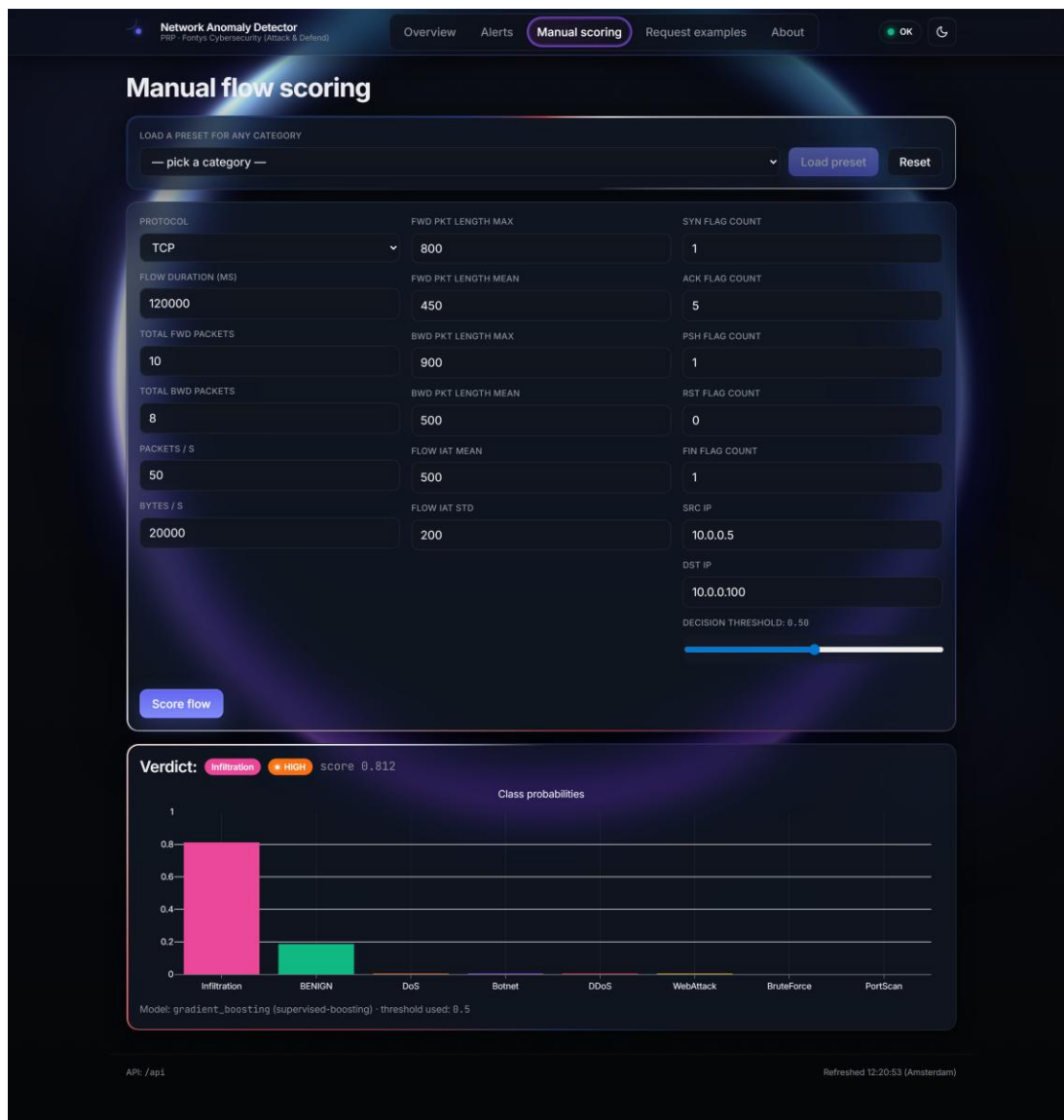


Figure 22 — Dark-theme manual scoring: the same hand-built flow returns an infiltration verdict and a full class-probability chart.

References

- [1] Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. ICISSP.
- [2] Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation Forest. IEEE International Conference on Data Mining.
- [3] Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly Detection: A Survey. ACM Computing Surveys, 41(3).
- [4] Ring, M., Wunderlich, S., Scheuring, D., Landes, D., & Hotho, A. (2019). A Survey of Network-based Intrusion Detection Data Sets. Computers & Security, 86.
- [5] Lashkari, A. H., et al. (2017). Characterization of Tor Traffic using Time-based Features (CICFlowMeter). ICISSP.
- [6] Scarfone, K., & Mell, P. (2007). Guide to Intrusion Detection and Prevention Systems (IDPS). NIST SP 800-94.
- [7] Microsoft. The STRIDE Threat Model. Microsoft Security Development Lifecycle.
- [8] MITRE ATT&CK Framework. <https://attack.mitre.org/>
- [9] OWASP API Security Top 10 (2023). <https://owasp.org/API-Security/>
- [10] OWASP Machine Learning Security Top 10 (2023). <https://owasp.org/www-project-machine-learning-security-top-10/>
- [11] scikit-learn documentation: Novelty and Outlier Detection. https://scikit-learn.org/stable/modules/outlier_detection.html
- [12] FastAPI documentation. <https://fastapi.tiangolo.com/>
- [13] Streamlit documentation. <https://docs.streamlit.io/>
- [14] CICIDS2017 Dataset. <https://www.unb.ca/cic/datasets/ids-2017.html>
- [15] DOT-framework -ICT Research Methods. <https://ictresearchmethods.nl/dot-framework/>